

RESEARCHPILOT

Investigation Report

For a binary classifier trained on synthetic data, adding a feature strongly correlated with the label during training improves in-distribution accuracy but reduces accuracy when that correlation reverses at test time

October 03, 2026

1 Conjecture

Compare two binary classifiers trained on synthetic data using the same learning procedure: a baseline using the original features and an augmented classifier also using an additional feature strongly correlated with the label in the training distribution. The claim is that the augmented classifier has strictly higher accuracy than the baseline on independent test data from the training distribution, but strictly lower accuracy than the baseline on test data where the added feature's label correlation reverses. Writing these accuracy differences as Δ_{ID} and Δ_{rev} , respectively, the claimed comparison is $\Delta_{ID} > 0$ and $\Delta_{rev} < 0$. The statement does not specify which classifiers, data generators, or correlation strengths it covers.

Working assumptions: User-given conditions: the task is binary classification on synthetic data; an added feature is strongly label-correlated during training; its label correlation reverses at test time; and the comparison concerns accuracy with versus without that feature., Source context: the supplied passages distinguish average in-distribution accuracy from worst-group accuracy and discuss models relying on spurious correlations. They do not specify a synthetic generator or a feature-addition experiment matching this claim., Chosen working conditions: isolate the intervention by retaining the same training examples, labels, original features, model family, regularization coefficient, and fitting tolerance across the two fits. Change only the availability of the added feature. At reversed testing, change only its conditional distribution given the label., Chosen initial numerical settings: use $\mu = 1$, $p = 0.95$, $\lambda = 0.01$, $n = 2000$, $N = 10000$, and $R = 30$, with an iteration cap of 100000. These values operationalize one test case rather than restricting the user's claim or guaranteeing its predictions., Chosen sampling conditions: training samples and both test samples are independent draws from their specified laws; different repeats are independent. The paired comparison within a repeat uses the same underlying training data and the same test examples for both classifiers. No test labels are used for fitting or hyperparameter selection.

Unverified supporting observations to check: Not specified.

Unverified contradicting observations to check: Not specified.

2 Assessment

ResearchPilot assessment: Both predicted accuracy comparisons are supported in the tested synthetic logistic-classification setting. The evidence establishes that the phenomenon occurs, but does not establish how typical it is across other classifiers, generators,

or correlation strengths. Related literature identifies qualifications or additional assumptions for the conjecture. Experiments support the conjecture in the tested cases.

I interpret the unspecified scope as a proposed empirical phenomenon, not as a universal guarantee for every binary classifier and synthetic distribution. Under that interpretation, Experiment 1: Feature augmentation under correlation reversal provides direct support for both comparisons.

All five independent repeats showed strictly higher in-distribution accuracy and strictly lower reversed-distribution accuracy for the augmented classifier. In-distribution accuracy ranged from 0.8373 to 0.8394 for the baseline and from 0.9576 to 0.9602 for the augmented classifier. Under reversal, the corresponding ranges were 0.8391–0.8418 and 0.2690–0.3457. The mean paired empirical differences were $\bar{\Delta}_P = 0.12120$ and $\bar{\Delta}_Q = -0.54152$. Thus, the experiment demonstrates degradation relative to the baseline under reversal, not merely a decline in the augmented classifier’s own accuracy.

The controls make this a meaningful feature-addition comparison. Both fits used the same training examples, original features, labels, regularization coefficient, and fitting tolerance. Both classifiers were evaluated on the same test examples within each repeat. Only the added feature’s conditional distribution changed under reversal; the original feature-label law remained fixed. The method was full-batch gradient descent for logistic empirical risk minimization with an L2 penalty, including intercept regularization. All ten fits converged, and no repeats were incomplete or flagged. The executed design contained five repeats, rather than the nominal initial thirty; the smaller count was a design revision, not execution truncation.

The tested construction used a Gaussian original-feature signal with $\mu = 1$, a binary added feature with $p = 0.95$, and regularization strength $\lambda = 0.01$. Added-feature agreement with the label changed from 0.95 to 0.05 under reversal. Each repeat used 2,000 training examples and 10,000 examples per test law. The reported across-repeat standard errors were approximately 0.000359 in distribution and 0.012554 under reversal, in accuracy proportions. These measurements support the observed signs, but finite-test estimates are not proofs of the corresponding population inequalities. No confidence intervals, hypothesis tests, or analytical population evaluations were supplied.

The inspected literature passages provide context rather than a directly matching confirmation. [Distributionally Robust Neural Networks for Group Shifts: On the Importance of Regularization for Worst-Case Generalization](#) documents high average accuracy alongside poor worst-group performance and improvements from regularized group DRO. Those comparisons change training methods, and worst-group accuracy is not average accuracy under a reversed test distribution. [Complexity Matters: Dynamics of Feature Learning in the Presence of Spurious Correlations](#) reports method-dependent core-feature learning after retraining, not the two paired accuracy differences. [SFP: Spurious Feature-targeted Pruning for Out-of-Distribution Generalization](#) describes constructed spurious-background benchmarks, but the supplied passage contains no matching accuracy comparison.

[Assessing Robustness to Spurious Correlations in Post-Training Language Models](#) reports stable or slightly improved accuracy in some setups as spuriousness increases. This qualifies a broad expectation of uniformly harmful spuriousness. It does not contradict the present experiment: increasing spuriousness in language-model post-training is different from reversing an added feature’s correlation in this paired binary-classification comparison.

Generality remains unresolved because only one generator, classifier family, and parameter configuration were tested. There was no sweep over stable-signal strength, correlation

strength, regularization, feature scale, or sample size. Unequal-magnitude reversal and conditional dependence between features were also untested. Redundant or ignored added features are relevant boundary cases for strict inequalities, but they are not supplied empirical counterexamples. These limits restrict extrapolation; they do not negate the clear occurrence observed in Experiment 1.

Direct experimental support: Experiment 1: Feature augmentation under correlation reversal found both predicted signs in all five repeats. In-distribution accuracy was 83.73–83.94% for the baseline and 95.76–96.02% for the augmented classifier. Under reversal, it was 83.91–84.18% and 26.90–34.57%, respectively. The recorded mean paired differences were $\overline{\Delta}_P = 0.12120$ and $\overline{\Delta}_Q = -0.54152$: a gain of 12.12 percentage points and a loss of 54.152 percentage points. Their across-repeat standard errors were approximately 0.000359 and 0.012554 in accuracy proportions.

Experimental scope and controls: Both models used the same training examples and paired test examples. Full-batch gradient descent fitted logistic empirical risk minimization with an L2 penalty, including intercept regularization. The Gaussian original-feature signal used $\mu = 1$; the binary added feature agreed with the label with probability 0.95 during training and 0.05 under reversal. Only the added feature’s conditional distribution changed. All ten fits converged, with no flagged or incomplete repeats. The resource-limited plan specified five repeats, replacing the nominal initial thirty; this was not execution truncation. These controls support the feature-availability comparison, but one generator and one regularization setting cannot establish a general accuracy guarantee. No confidence interval or hypothesis test was reported.

Related literature: The inspected passages, rather than merely abstracts, provide context but no direct test of both strict inequalities. [Distributionally Robust Neural Networks for Group Shifts: On the Importance of Regularization for Worst-Case Generalization](#) reports high average accuracy alongside poor worst-group performance and improvements from regularized group DRO. Its comparisons change training methods, not feature availability, and worst-group accuracy is not reversed-distribution average accur

3 Literature evidence

3.1 Supporting evidence

- None verified.

3.2 Contradictory evidence

- None verified.

3.3 Important qualifications

- performance does not invariably decline as the amount of spurious data increases: some setups show stable or even slightly improved accuracy at higher spuriousness levels. [Assessing Robustness to Spurious Correlations in Post-Training Language Models], section-0, page 2; empirical_result. Assumptions: The source’s language-model post-training setups., The comparison increases the amount of spurious data rather than explicitly reversing correlation., The supplied limitations restrict evaluation to Llama

3.x instruction-tuned models.. This introductory report qualifies a broader expectation of uniformly harmful spuriousness. It is not a counterexample to the exact binary feature-addition conjecture, and the supplied passage gives n

3.4 Related evidence

- The value here denotes core correlation after retraining. It is observed that previous debiasing algorithms either failed on the designed spurious task or cause failure to core feature learning. [Complexity Matters: Dynamics of Feature Learning in the Presence of Spurious Correlations], section-0, page 45; empirical_result. Assumptions: The designed HardStaircase and HardDomino experiments and the debiasing methods listed in Table 6., The reported outcome is core correlation after retraining., The caption specifies clean training with $\lambda = 0.5$ and original training with $\lambda = 0.95$ for HardDomino and $\lambda = 0.9$ for the staircase task.. This reports method-dependent feature-learning outcomes, not the signs of Δ_{ID} or Δ_{rev} . The supplied passage does not define λ sufficiently to identify it with the conjecture’s correlation strength.
- We evaluate the proposed SFP on three constructed OOD datasets, including Full-colored-mnist, Colored-object, and Scene-object. As shown in Fig. 3, the invariant features are the focused digits or objects in the foreground, and the spurious features are the background scene. [SFP: Spurious Feature-targeted Pruning for Out-of-Distribution Generalization], section-0, page 8; discussion. Assumptions: The proposed SFP method is evaluated on the three named constructed OOD datasets., Foreground digits or objects are designated invariant features; backgrounds are designated spurious features.. This supplies a relevant way to construct spurious-feature benchmarks, but gives neither accuracy results nor the conjecture’s same-procedure feature-addition comparison.

4 Computational investigation

Experiments planned: 1. Executed successfully: 1. Stochastic trials: 0.

4.1 Experiment 1: Feature augmentation under correlation reversal

Test type: falsifying. **Baselines:** Original-feature logistic classifier **Metrics:** Test accuracy, in proportions: $\hat{A}_{D,r}(f_j) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{f_j(z_{D,r,i}^{(j)}) = Y_{D,r,i}\}$, where $D \in \{P, Q\}$, $j \in \{0, 1\}$, and $N = n_{\text{test}}$. Paired augmentation difference, in accuracy proportions: $\hat{\Delta}_{D,r} = \hat{A}_{D,r}(f_1) - \hat{A}_{D,r}(f_0)$. Report every completed repeat separately., Across-seed mean and standard error, in accuracy proportions: $\bar{\Delta}_D = \frac{1}{K} \sum_{r \in \mathcal{C}} \hat{\Delta}_{D,r}$ and $\text{SE}_D = \sqrt{\frac{1}{K(K-1)} \sum_{r \in \mathcal{C}} (\hat{\Delta}_{D,r} - \bar{\Delta}_D)^2}$, where $\mathcal{C}$ contains repeats with both fits converged and both test evaluations completed, and $K = |\mathcal{C}|$. Leave the mean undefined when no repeat qualifies and the standard error undefined when fewer than two qualify., Fit diagnostics: final gradient norm, number of gradient updates, convergence status, and iteration-cap status for each fit; total elapsed wall-clock time in seconds and count of incomplete repeats.

Algorithm pseudocode:

1. Start a wall-clock timer and process seeds in their listed order, checking the global time cap before each fit update and test evaluation.

2. For each seed, initialize an independent random-number stream and draw n_{train} independent training triples using the specified generator: $Y_i \sim \text{Uniform}(\{-1, +1\})$, $\varepsilon_i \sim \mathcal{N}(0, 1)$, $B_i \sim \text{Bernoulli}(p)$, $X_i = \mu Y_i + \varepsilon_i$, and $S_i = Y_i(2B_i - 1)$, with all primitive draws independent.
3. Construct the baseline design rows $z_i^{\{0\}} = (1, X_i)^{\text{top}}$ and augmented design rows $z_i^{\{1\}} = (1, X_i, S_i)^{\text{top}}$ from the same training triples.
4. For each model, initialize all coefficients to `initial_parameter` and compute $\eta_j = \left(\lambda + \frac{\|Z_j\|_2^2}{4n_{\text{train}}} \right)^{-1}$, obtaining the squared spectral norm from the largest eigenvalue of the small Gram matrix.
5. At each fitting iteration, compute the full-batch gradient $g_j^{\{t\}} = -\frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \frac{Y_i z_i^{\{j\}}}{1 + \exp(-\left(Y_i (\theta_j^{\{t\}})^{\text{top}} z_i^{\{j\}}\right))} + \lambda \theta_j^{\{t\}}$, evaluating the logistic factor with a numerically stable implementation.
6. If the gradient norm meets `gradient_tolerance`, retain the current coefficients and mark the fit converged; otherwise, if the update count reaches `iteration_cap_per_fit`, retain the coefficients and flag the fit.
7. For a fit not yet stopped, apply $\theta_j^{\{t+1\}} = \theta_j^{\{t\}} - \eta_j g_j^{\{t\}}$ and return to the gradient computation.
8. After fitting both models, independently generate n_{test} fresh triples for each test law using the training generator with $p_P = p$ and $p_Q = 1 - p$; keep both test samples independent of training and of each other.
9. For each test law, evaluate both models on the same triples, predicting the positive label when the fitted linear score is nonnegative and the negative label otherwise.
10. Compute the defined test accuracies and paired augmentation differences, and record the fit diagnostics for the repeat.
11. If the global time cap is reached, mark the current unfinished repeat incomplete and stop without starting additional work.
12. Aggregate the paired differences over qualifying completed seeds using the defined mean and standard error, retaining flagged-fit results separately and reporting total elapsed time.

Working assumptions:

- User-given conditions: the task is binary classification on synthetic data; an added feature is strongly label-correlated during training; its label correlation reverses at test time; and the comparison concerns accuracy with versus without that feature.
- Source context: the supplied passages distinguish average in-distribution accuracy from worst-group accuracy and discuss models relying on spurious correlations. They do not specify a synthetic generator or a feature-addition experiment matching this claim.
- Chosen working conditions: isolate the intervention by retaining the same training examples, labels, original features, model family, regularization coefficient, and fitting tolerance across the two fits. Change only the availability of the added feature. At reversed testing, change only its conditional distribution given the label.
- Chosen initial numerical settings: use $\mu = 1$, $p = 0.95$, $\lambda = 0.01$, $n = 2000$, $N = 10000$, and $R = 30$, with an iteration cap of 100000. These values operationalize one test case rather than restricting the user's claim or guaranteeing its predictions.
- Chosen sampling conditions: training samples and both test samples are independent draws from their specified laws; different repeats are independent. The paired compar-

ison within a repeat uses the same underlying training data and the same test examples for both classifiers. No test labels are used for fitting or hyperparameter selection.

- The resource-limited repeat count replaces the nominal initial repeat count; the generator, learning procedure, and other initial numerical settings remain unchanged. **Exact planned settings:**
- varied parameters: {}
- held constant settings: {"mu":1, "p":0.95, "lambda":0.01, "n_train":2000, "n_test":10000, "repeat_count":5, "initial_parameter":0, "gradient_tolerance":1e-06, "iteration_cap_per_fit":100000, "training_law":"P", "test_laws":["P", "Q"], "arithmetic":"float64"}
- stopping rule: "Stop each fit when its gradient norm is at most gradient_tolerance; otherwise flag it at iteration_cap_per_fit. Stop the entire experiment at the wall-clock cap, marking unfinished repeats incomplete."
- work cap: {"max_experiment_seconds":300, "maximum_fits":10, "maximum_gradient_updates":1000000}
- sweep: "One fixed parameter configuration; no hyperparameter selection."
- random seeds: [17, 43, 101, 229, 503].

Execution: completed; run experiment_c20382037202; runtime 7.86 seconds. **Recorded configuration:**

```
{
  "varied_parameters": {},
  "held_constant_settings": {
    "mu": 1, "p": 0.95, "lambda": 0.01, "n_train": 2000, "n_test": 10000,
    "repeat_count": 5, "initial_parameter": 0, "gradient_tolerance": 1e-06,
    "iteration_cap_per_fit": 100000, "training_law": "P", "test_laws": ["P", "Q"],
    "arithmetic": "float64",
    "stopping_rule": "Stop each fit when its gradient norm is at most gradient_tolerance; otherwise flag it at iteration_cap_per_fit. Stop the entire experiment at the wall-clock cap, marking unfinished repeats incomplete.",
    "work_cap": {
      "max_experiment_seconds": 300, "maximum_fits": 10, "maximum_gradient_updates": 1000000
    },
    "sweep": "One fixed parameter configuration; no hyperparameter selection."
  }
}

Recorded metrics: {
  "design": "Feature augmentation under correlation reversal",
  "method": "Full-batch gradient descent for L2-regularized logistic ERM; intercept also regularized",
  "settings": {
    "mu": 1, "p": 0.95, "lambda": 0.01, "n_train": 2000, "n_test": 10000, "repeat_count": 5, "initial_parameter": 0, "gradient_tolerance": 1e-06, "iteration_cap_per_fit": 100000, "training_law": "P", "test_laws": ["P", "Q"], "arithmetic": "float64",
    "seeds": [17, 43, 101, 229, 503], "wall_clock_cap_seconds": 300.0, "paired_comparisons": {
      "P": {
        "seeds": [17, 43, 101, 229, 503], "control": [0.8373, 0.8381, 0.8391, 0.8394, 0.8379], "treatment": [0.9576, 0.9602, 0.9599, 0.9602, 0.9599],
        "higher_supports": true, "control_censored": [false, false, false, false, false], "treatment_censored": [false, false, false, false, false],
        "censoring_meaning": "True denotes a completed evaluation with at least one nonconverged fit; no accuracy imputation."
      }, "Q": {
        "seeds": [17, 43, 101, 229, 503], "control": [0.8391, 0.8418, 0.8396, 0.8418, 0.8406], "treatment": [0.269, 0.3457, 0.2916, 0.2857, 0.3033],
        "higher_supports": false, "control_censored": [false, false, false, false, false], "treatment_censored": [false, false, false, false, false],
        "censoring_meaning": "True denotes a completed evaluation with"
      }
    }
  }
}
```

```

at least one nonconvergedfit;no accuracy imputation."}}, "repeats":
[{"seed":17, "completed":true, "qualifying":true, "fits":[{"coefficie
nts":[-0.036722249788438424,1.7505027360569685], "gradient_norm":
8.398832906697671e-07, "gradient_updates":54, "converged":true,
"iteration_cap_status":false, "time_cap_status":false, "status":
"converged", "step_size":1.9442706246177044, "squared_spectral_norm":
4034.653535730404, "model":"baseline"}, {"coefficients":[-0.0340658865
5859461,1.3479319674307417,2.3374057667171395], "gradient_norm":
9.459571654933459e-07, "gradient_updates":136, "converged":true,
"iteration_cap_status":false, "time_cap_status":false, "status":
"converged", "step_size":1.5535646762629585, "squared_spectral_norm":
5069.44766847023, "model":"augmented"}], "tests":{"P":{"n":10000,
"correct_counts":[8373, 9576], "accuracy":[0.8373, 0.9576], "difference"
:0.12029999999999996, "correctness_table":[[112, 1515], [312, 8061]]}
, "Q":{"n":10000, "correct_counts":[8391, 2690], "accuracy":[0.8391,
0.269], "difference":-0.5700999999999999, "correctness_table":[[1522,
87], [5788, 2603]]}}, "status":"completed"}, {"seed":43, "completed":
true, "qualifying":true, "fits":[{"coefficients":[0.014439606
808494283,1.8923197288567828], "gradient_norm":8.5874938479
55172e-07, "gradient_updates":62, "converged":true, "iteration_cap_
status":false, "time_cap_status":false, "status":"converged", "step_
size":1.9285722335192441, "squared_spectral_norm":4068.1464167
933495, "model":"baseline"}, {"coefficients":[-0.02291728306414449,
1.4276285334926944,2.131021334737412], "gradient_norm":9.4238128278
60345e-07, "gradient_updates":132, "converged":true, "iteration_cap_
status":false, "time_cap_status":false, "status":"converged", "step_
size":1.5122960759864659, "squared_spectral_norm":5209.969422675137,
"model":"augmented"}], "tests":{"P":{"n":10000, "correct_counts":
[8381, 9602], "accuracy":[0.8381, 0.9602], "difference":0.1221000000
000001, "correctness_table":[[143, 1476], [255, 8126]]}, "Q":{"n":
10000, "correct_counts":[8418, 3457], "accuracy":[0.8418, 0.3457],
"difference":-0.4961, "correctness_table":[[1504, 78], [5039, 3379]]}}
, "status":"completed"}, {"seed":101, "completed":true, "qualifying"
:true, "fits":[{"coefficients":[-0.020304963874255007,1.6820890252
948368], "gradient_norm":8.225179981235056e-07, "gradient_updates":
50, "converged":true, "iteration_cap_status":false, "time_cap_status":
false, "status":"converged", "step_size":1.9161528167364024, "squared_
spectral_norm":4095.0323513474386, "model":"baseline"}, {"coefficien
ts":[-0.031479544497562344,1.3322343787234365,2.19830144871433]
, "gradient_norm":9.760013126326063e-07, "gradient_updates":124,
"converged":true, "iteration_cap_status":false, "time_cap_status":
false, "status":"converged", "step_size":1.5444410849289982, "squared_
spectral_norm":5099.867382489232, "model":"augmented"}], "tests":
{"P":{"n":10000, "correct_counts":[8391, 9599], "accuracy":[0.8391,
0.9599], "difference":0.120800000000000002, "correctness_table":
[[111, 1498], [290, 8101]]}, "Q":{"n":10000, "correct_counts":[8396,
2916], "accuracy":[0.8396, 0.2916], "difference":-0.548, "correctness
_table":[[1530, 74], [5554, 2842]]}}, "status":"completed"}, {"seed":
229, "completed":true, "qualifying":true, "fits":[{"coefficients":
[-0.09869393444969286,1.6953885437347203], "gradient_norm":
8.914701992025213e-07, "gradient_updates":51, "converged":true,

```

```

"iteration_cap_status":false,"time_cap_status":false,"status":
"converged","step_size":1.9178470722106316,"squared_spectral_
norm":4091.3440638302272,"model":"baseline"}, {"coefficients":
[-0.07519058784082672,1.2961440736551078,2.189967238466383],
"gradient_norm":9.86821352078639e-07,"gradient_updates":122,
"converged":true,"iteration_cap_status":false,"time_cap_status":
false,"status":"converged","step_size":1.5322316301934225,"squared_
spectral_norm":5141.142706073829,"model":"augmented"}], "tests":
{"P":{"n":10000,"correct_counts":[8394,9602],"accuracy":[0.8394,
0.9602],"difference":0.120800000000000002,"correctness_table":[[108,
1498],[290,8104]]}, "Q":{"n":10000,"correct_counts":[8418,2857]
,"accuracy":[0.8418,0.2857],"difference":-0.5561,"correctness_
table":[[1489,93],[5654,2764]]}}, "status":"completed"}, {"seed":
503,"completed":true,"qualifying":true,"fits":[{"coefficients":[-
0.02114649171564222,1.751933768942262],"gradient_norm":9.6372017892
11533e-07,"gradient_updates":47,"converged":true,"iteration_cap_
status":false,"time_cap_status":false,"status":"converged","step_
size":2.130985176208863,"squared_spectral_norm":3674.1321682
173448,"model":"baseline"}, {"coefficients":[0.03268745237851052,
1.3516363518539756,2.152294730843465],"gradient_norm":9.1708020738
95643e-07,"gradient_updates":113,"converged":true,"iteration_cap_
status":false,"time_cap_status":false,"status":"converged","step_
size":1.6644056531207556,"squared_spectral_norm":4726.5205648
63514,"model":"augmented"}], "tests":{"P":{"n":10000,"correct_
counts":[8379,9599],"accuracy":[0.8379,0.9599],"difference":0.122,
"correctness_table":[[116,1505],[285,8094]]}, "Q":{"n":10000,
"correct_counts":[8406,3033],"accuracy":[0.8406,0.3033],"difference"
:-0.5373,"correctness_table":[[1514,80],[5453,2953]]}}, "status":
"completed"}], "summary":{"P":{"K":5,"mean_difference":0.1212000000
0000002,"standard_error":0.00035916569992137395,"qualifying_seeds":
[17,43,101,229,503],"differences":[0.12029999999999996,0.1221000000
000001,0.12080000000000002,0.12080000000000002,0.122]},"Q":{"K":
5,"mean_difference":-0.54152,"standard_error":0.012554218414540987,
"qualifying_seeds":[17,43,101,229,503],"differences":[-0.5700999999
999999,-0.4961,-0.548,-0.5561,-0.5373]}}, "flagged_completed_seeds":
[], "incomplete_repeats":0, "unstarted_seeds":[], "elapsed_seconds":
0.9635906209999998, "scope":"One fixed synthetic setup; not a
universal test over classifier families or generators.", "correctness_
_table_axes":["baseline correct:0,1","augmented correct:0,1"]}

```

Finding (supports): The recorded comparisons support both predicted signs in this specific logistic-classification setup. Across all five seeds, in-distribution accuracy was 0.8373–0.8394 for the baseline and 0.9576–0.9602 for the augmented classifier. The mean paired gain was $\bar{\Delta}_P = 0.12120$, or 12.12 percentage points. Under correlation reversal, baseline accuracy was 0.8391–0.8418 versus 0.2690–0.3457 for the augmented classifier. The mean paired difference was $\bar{\Delta}_Q = -0.54152$, or a loss of 54.152 percentage points. Every repeat had a strictly positive in-distribution difference and a strictly negative reversed-distribution difference. This supports the conjecture for the tested setup, not a universal claim over classifiers and generators.

Uncertainty: Only five independent training repeats were evaluated, each using 2,000 training examples and 10,000 test examples per test law. The recorded across-seed standard errors of the mean accuracy differences were approximately 0.00035917 in distribution and 0.01255422 under reversal, in accuracy proportions. These estimates reflect training and

finite-test sampling variation; no hypothesis test or confidence interval was reported. All repeats completed with qualifying fits, with no flagged completed **Robustness**: Both predicted signs persisted across seeds 17, 43, 101, 229, and 503. Seed replication was the only robustness check. The experiment used one Gaussian original-feature generator and one binary added-feature generator, with $\mu = 1$, $p = 0.95$, and $\lambda = 0.01$, trained using full-batch gradient descent for L2-regularized logistic empirical risk minimization, including intercept regularization. No alternative classifier families, generators, signal strengths, correlation strengths, regular **Result JSON fields used**: /summary/P, /summary/Q, /paired_comparisons/P/control, /paired_comparisons/P/treatment, /paired_comparisons/Q/control, /paired_comparisons/Q/treatment, /settings, /method, /flagged_completed_seeds, /incomplete_repeats **Artifacts**: result.json, visualization.svg

Exact reusable experiment and visualization code is available on the Experiments page.

```
# RESEARCHPILOT EXPERIMENT PLAN

# BASELINES AND METRICS

# ['Original-feature logistic classifier']

# ['Test accuracy, in proportions:  $\frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\widehat{f}_j(z_{D,r,i}) = Y_{D,r,i}\}$ , where  $D \in \{P, Q\}$ ,  $j \in \{1, \dots, J\}$ ']

# WORKING ASSUMPTIONS

# ['User-given conditions: the task is binary classification on synthetic data; an added feature is strongly label-correlated during training; its label correlation reverses at test time; and the comparison']

# ALGORITHM STEPS

# ['Start a wall-clock timer and process `seeds` in their listed order, checking the global time cap before each fit update and test evaluation.', 'For each seed, initialize an independent random-number']

# SETTINGS AND RANDOM SEEDS

# {'varied_parameters': {}, 'held_constant_settings': {'mu': 1, 'p': 0.95, 'lambda': 0.01, 'n_train': 2000, 'n_test': 10000, 'repeat_count': 5, 'initial_parameter': 0, 'gradient_tolerance': 1e-06, 'iter': 10000}}

# [17, 43, 101, 229, 503]

# EXECUTABLE STUDY

import json
import math
import random
import time

# Fixed operational test: regularized logistic ERM, not group DRO.
# The resource-limited five repeats supersede the inherited nominal R=30.
START = time.perf_counter()
CAP = 300.0
SEEDS = [17, 43, 101, 229, 503]
SETTINGS = dict(mu=1, p=0.95, **{'lambda': 0.01}, n_train=2000, n_test=10000, repeat_count=5, initial_parameter=0,
```

```

        gradient_tolerance=1e-6, iteration_cap_per_fit=100000,
        training_law='P', test_laws=['P', 'Q'], arithmetic=
'float64')

class TimeCap(Exception):
    pass

def check():
    if time.perf_counter() - START >= CAP:
        raise TimeCap()

def sample(rng, n, p):
    data = []
    for i in range(n):
        if i % 128 == 0:
            check()

        y = 1.0 if rng.random() < 0.5 else -1.0
        eps = rng.gauss(0.0, 1.0)
        b = rng.random() < p
        data.append((y + eps, y * (1.0 if b else -1.0), y))
    return data

def largest_eigenvalue(a):
    # Symmetric Jacobi diagonalization of the 2x2 or 3x3 Gram matrix.
    a = [row[:] for row in a]
    d = len(a)
    for sweep in range(100):
        check()
        p, q = max(((i, j) for i in range(d) for j in range(i+1, d)),
                    key=lambda ij: abs(a[ij[0]][ij[1]]))
        off = a[p][q]
        if abs(off) <= 1e-14 * max(1.0, max(abs(a[i][i]) for i in range(d)
)):
            return max(a[i][i] for i in range(d))
        tau = (a[q][q] - a[p][p]) / (2.0 * off)
        t = math.copysign(1.0, tau) / (abs(tau) + math.hypot(1.0, tau))
        c = 1.0 / math.sqrt(1.0 + t*t)
        s = t*c
        app, aqq = a[p][p], a[q][q]
        a[p][p] = app - t*off
        a[q][q] = aqq + t*off
        a[p][q] = a[q][p] = 0.0

```

```

    for k in range(d):
        if k != p and k != q:
            akp, akq = a[k][p], a[k][q]
            a[k][p] = a[p][k] = c*akp - s*akq
            a[k][q] = a[q][k] = s*akp + c*akq
    raise ArithmeticError('Jacobi diagonalization did not converge')

def logistic_factor(m):
    if m >= 0.0:
        e = math.exp(-m)
        return e / (1.0 + e)
    return 1.0 / (1.0 + math.exp(m))

def fit(data, augmented):
    d = 3 if augmented else 2
    theta = [0.0]*d
    diag = dict(coefficients=theta, gradient_norm=None, gradient_updates=
0,
                converged=False, iteration_cap_status=False, time_cap_
status=False,
                status='initializing', step_size=None, squared_spectral_
norm=None)
    try:
        check()
        gram = [[0.0]*d for _ in range(d)]
        for x, s, y in data:
            z = [1.0, x, s][:d]
            for k in range(d):
                for l in range(d):
                    gram[k][l] += z[k]*z[l]
        eig = largest_eigenvalue(gram)
        eta = 1.0 / (0.01 + eig/(4.0*len(data)))
        diag.update(step_size=eta, squared_spectral_norm=eig)
        updates = 0
        while True:
            check()
            a, b = theta[:2]
            c = theta[2] if augmented else 0.0
            g0 = g1 = g2 = 0.0
            for x, s, y in data:
                u = -y * logistic_factor(y*(a + b*x + c*s))

```

```

        g0 += u
        g1 += u*x
        if augmented:
            g2 += u*s
    n = len(data)
    g = [g0/n + 0.01*a, g1/n + 0.01*b]
    if augmented:
        g.append(g2/n + 0.01*c)
    norm = math.sqrt(sum(v*v for v in g))
    diag['gradient_norm'] = norm
    if norm <= 1e-6:
        diag.update(converged=True, status='converged')
        break
    if updates >= 100000:
        diag.update(iteration_cap_status=True, status='iteration_
cap')
        break
    check() # Mandatory global-cap check before every gradient
update.

    theta = [v - eta*w for v, w in zip(theta, g)]
    updates += 1
    diag.update(coefficients=theta, gradient_updates=updates,
gradient_norm=None)
except TimeCap:
    diag.update(time_cap_status=True, status='time_cap')
return diag

def evaluate(data, fits):
    check()
    counts = [0, 0]
    # Discordance counts retain information about within-test pairing.
    joint = [[0, 0], [0, 0]]
    t0, t1 = [f['coefficients'] for f in fits]
    for i, (x, s, y) in enumerate(data):
        if i % 128 == 0:
            check()

            c0 = int((1.0 if t0[0] + t0[1]*x >= 0 else -1.0) == y)
            c1 = int((1.0 if t1[0] + t1[1]*x + t1[2]*s >= 0 else -1.0) == y)
            counts[0] += c0
            counts[1] += c1

```

```

        joint[c0][c1] += 1
    check()
    acc = [v/len(data) for v in counts]
    return dict(n=len(data), correct_counts=counts, accuracy=acc,
                difference=acc[1]-acc[0], correctness_table=joint)

repeats = []
for seed in SEEDS:
    rec = dict(seed=seed, completed=False, qualifying=False, fits=[],
               tests={})
    try:
        check()
        repeats.append(rec)
        rng = random.Random(seed)
        train = sample(rng, 2000, 0.95)
        for augmented in (False, True):
            check()
            f = fit(train, augmented)
            f['model'] = 'augmented' if augmented else 'baseline'
            rec['fits'].append(f)
            if f['time_cap_status']:
                raise TimeCap()

        # Both test samples are fresh and independent, using the same
        seed stream.
        tests = {'P': sample(rng, 10000, 0.95), 'Q': sample(rng, 10000,
0.05)}

        for law in ('P', 'Q'):
            check()
            rec['tests'][law] = evaluate(tests[law], rec['fits'])
        rec['completed'] = True
        rec['qualifying'] = all(f['converged'] for f in rec['fits'])
        rec['status'] = 'completed' if rec['qualifying'] else 'completed_
flagged_fit'
    except TimeCap:
        if rec not in repeats:
            repeats.append(rec)
        rec['status'] = 'incomplete_time_cap'
        break

qualified = [r for r in repeats if r['qualifying']]
completed = [r for r in repeats if r['completed']]

```

```

summary = {}
comparisons = {}
for law in ('P', 'Q'):
    values = [r['tests'][law]['difference'] for r in qualified]
    k = len(values)
    mean = sum(values)/k if k else None
    se = math.sqrt(sum((v-mean)**2 for v in values)/(k*(k-1))) if k > 1
    else None
    summary[law] = dict(K=k, mean_difference=mean, standard_error=se,
                        qualifying_seeds=[r['seed'] for r in qualified],
differences=values)
    comparisons[law] = dict(seeds=[r['seed'] for r in completed],
                            control=[r['tests'][law]['accuracy'][0] for r in completed],
                            treatment=[r['tests'][law]['accuracy'][1] for r in completed],
                            higher_supports=(law == 'P'),
                            control_censored=[not r['qualifying'] for r in completed],
                            treatment_censored=[not r['qualifying'] for r in completed],
                            censoring_meaning='True denotes a completed evaluation with at
least one nonconverged fit; no accuracy imputation.')

result = dict(design='Feature augmentation under correlation reversal',
              method='Full-batch gradient descent for L2-regularized logistic ERM;
intercept also regularized',
              settings=SETTINGS, seeds=SEEDS, wall_clock_cap_seconds=CAP,
              paired_comparisons=comparisons, repeats=repeats, summary=summary,
              flagged_completed_seeds=[r['seed'] for r in completed if not
r['qualifying']],
              incomplete_repeats=sum(not r['completed'] for r in repeats),
              unstarted_seeds=[s for s in SEEDS if s not in [r['seed'] for r in
repeats]],
              elapsed_seconds=time.perf_counter()-START,
              scope='One fixed synthetic setup; not a universal test over classifier
families or generators.',
              correctness_table_axes=['baseline correct: 0,1', 'augmented correct:
0,1'])

blob = json.dumps(result, ensure_ascii=False, allow_nan=False,
separators=(',', ':'))
assert len(blob.encode('utf-8')) <= 100000
with open('result.json', 'w', encoding='utf-8') as f:
    f.write(blob)

```

Visualization code (reads result.json):

```
import json
```

```

from html import escape

with open('result.json', encoding='utf-8') as f:
    r = json.load(f)

parts = ['<svg xmlns="http://www.w3.org/2000/svg" width="1000" height="760" viewBox="0 0 1000 760" role="img" aria-label="Paired classifier accuracy and augmentation differences">',

        '<title>Feature augmentation under correlation reversal</title>',

        '<desc>Top panels show paired test accuracies. Bottom panels show augmentation minus baseline by seed and qualifying mean plus or minus one across-seed standard error. Hollow markers denote flagged fits.</desc>',

        '<rect x="0" y="0" width="1000" height="760" fill="white"/>']

def text(x, y, s, size=14, anchor='start', color='#222222'):
    parts.append('<text x="%s" y="%s" font-family="sans-serif" font-size="%s" text-anchor="%s" fill="%s">%s</text>' % (x, y, size, anchor, color, escape(str(s))))

def line(x1, y1, x2, y2, color='aaaaaa', width=1, dash=None):
    a = ' stroke-dasharray="%s"' % dash if dash else ''
    parts.append('<line x1="%s" y1="%s" x2="%s" y2="%s" stroke="%s" stroke-width="%s"%s/>' % (x1, y1, x2, y2, color, width, a))

def circle(x, y, color, hollow=False, radius=4):
    parts.append('<circle cx="%s" cy="%s" r="%s" fill="%s" stroke="%s" stroke-width="2"/>' % (x, y, radius, 'white' if hollow else color, color))

text(500, 30, 'Feature augmentation under correlation reversal', 22, 'middle')

text(500, 54, 'Logistic ERM | paired training and test examples | fixed synthetic setup', 14, 'middle')

colors = {'P': '#2166ac', 'Q': '#b2182b'}

for col, law in enumerate(('P', 'Q')):
    left, right = 80+col*490, 440+col*490
    c = r['paired_comparisons'][law]
    text((left+right)/2, 85, 'P: in-distribution' if law=='P' else 'Q: reversed correlation', 18, 'middle')
    top, bottom = 110, 310
    def ay(v):
        return bottom-(bottom-top)*v
    for tick in (0, 0.25, 0.5, 0.75, 1):
        y = ay(tick)
        line(left, y, right, y, '#dddddd')
        text(left-10, y+4, '%g'%tick, 12, 'end')
    text(left, 102, 'Accuracy (proportion)', 12)

```

```

xs = [left+90,right-90]
for i, (a,b) in enumerate(zip(c['control'],c['treatment'])):
    offset = (i-(len(c['seeds'])-1)/2)*9
    hollow = c['control_censored'][i] or c['treatment_censored'][i]
    line(xs[0]+offset, ay(a), xs[1]+offset, ay(b), '#bcbbc')
    circle(xs[0]+offset, ay(a), '#444444', hollow)
    circle(xs[1]+offset, ay(b), colors[law], hollow)
for x,label in zip(xs, ('Baseline', 'Augmented')):
    text(x, 334, label, 14, 'middle')
text((left+right)/2, 375, 'Augmentation difference by seed', 16, 'middle')
top2, bottom2 = 405, 640
def dy(v):
    return bottom2-(v+1)*(bottom2-top2)/2
for tick in (-1,-0.5,0,0.5,1):
    y = dy(tick)
    line(left,y,right,y, '#777777' if tick==0 else '#dddddd', 1)
    text(left-10, y+4, '%g'%tick, 12, 'end')
text(left, 397, 'Accuracy difference (proportion)', 12)
n = len(c['seeds'])
for i, seed in enumerate(c['seeds']):
    x = left+25+i*max(1, (right-left-100)/max(1, n-1))
    v = c['treatment'][i]-c['control'][i]
    circle(x, dy(v), colors[law], c['control_censored'][i])
    text(x, 660, seed, 12, 'middle')
s = r['summary'][law]
if s['mean_difference'] is not None:
    x = right-12
    m, se = s['mean_difference'], s['standard_error']
    if se is not None:
        line(x, dy(m-se), x, dy(m+se), colors[law], 3)
        line(x-6, dy(m-se), x+6, dy(m-se), colors[law], 2)
        line(x-6, dy(m+se), x+6, dy(m+se), colors[law], 2)
        circle(x, dy(m), colors[law], False, 6)
        text(x, 660, 'Mean', 12, 'middle')
        text((left+right)/2, 690, 'K=%d; mean=%+.4f; SE=%s' % (s['K'], m,
'undefined' if se is None else '%.4f'%se), 13, 'middle')
    else:
        text((left+right)/2, 690, 'No qualifying mean; K=0', 13, 'middle')
text((left+right)/2, 714, 'Claimed direction: '+'(above zero' if law==

```

```
'P' else 'below zero'),13,'middle')

text(500,742,'Seed labels; mean bars =  $\pm 1$  SE. Hollow = flagged fit.
Incomplete repeats: %d; elapsed: %.2f s.' % (r['incomplete_repeats'],
r['elapsed_seconds']),12,'middle')

parts.append('</svg>')

svg = '\n'.join(parts)

assert len(svg.encode('utf-8')) <= 1000000

with open('visualization.svg','w',encoding='utf-8') as f:
    f.write(svg)
```

4.2 Potential counterexamples

- None recorded.

5 Interpretation

For the balanced-label synthetic generator with $X = Y + \varepsilon$, where $\varepsilon \sim \mathcal{N}(0, 1)$ is independent of the label, and a binary added feature independently agreeing with the label with probability 0.95 conditional on the label, logistic empirical risk minimization with $\lambda = 0.01$, including intercept regularization, is expected to have higher mean in-distribution accuracy after feature augmentation and lower mean accuracy when agreement falls to 0.05 while the original feature-label law remains fixed. This is a setting-specific empirical conjecture supported by five paired repeats with 2,000 training examples, not a guarantee for every fitted model or test sample.

6 What remains uncertain

- The statement does not specify whether it describes one intended setup, a typical empirical tendency, or every binary classifier and synthetic-data distribution. These scopes are materially different.
- The original synthetic generator, the strength and form of the stable predictive signal, the classifier family, the training objective, and the optimization settings are unspecified. Logistic empirical risk minimization is an explicit working choice, not a recovered user-specified algorithm.
- Strong correlation is not assigned a numerical threshold or a correlation measure. Pearson correlation is used in the chosen binary-feature construction; other feature types could require a different operational definition.
- Correlation reversal could mean equal-magnitude sign reversal, any sign change, or a reversal accompanied by other distribution changes. The chosen experiment uses equal-magnitude reversal and holds the original feature-label law fixed.
- The statement does not say whether accuracy is conditional on one fitted model, averaged over training randomness, or measured on a particular finite test set. The proposed repeated experiment estimates a training-averaged comparison.
- The supplied paper's worst-group comparison is not the same as average accuracy under a reversed test distribution. The passages do not establish that the user's claim refers to group DRO, nor do they supply the full updates of its referenced algorithm.

- Is the intended claim existential, a typical empirical tendency, or universal? The supplied evidence establishes occurrence in one setting but not its prevalence.
- Do both signs persist across stable-signal strengths, added-feature correlation strengths, regularization coefficients, feature scales, sample sizes, and classifier families? No parameter sweep was performed.
- What happens when the added feature is redundant or the learning procedure ignores it? These are untested boundary cases for strict improvement and strict degradation, not supplied empirical counterexamples.
- Do the population accuracy differences, conditional on each fit and averaged over training randomness, have the predicted signs? Larger independent replication, paired uncertainty intervals, or analytical evaluation would better separate these targets from finite-test observations.
- Does the comparison persist under unequal-magnitude reversal or dependence between the original and added features conditional on the label? The tested construction used equal-magnitude reversal and conditional independence.
- Can a directly matching literature study or theorem establish the paired feature-addition comparison? The inspected passages instead concern group robustness, re-training, pruning, or language-model post-training.

7 Sources

- [Complexity Matters: Dynamics of Feature Learning in the Presence of Spurious Correlations.](#)
- [SFP: Spurious Feature-targeted Pruning for Out-of-Distribution Generalization.](#)
- [Assessing Robustness to Spurious Correlations in Post-Training Language Models.](#)

8 Usage

Live model requests: 10. Reused model responses: 0. All cache hits: 0. Tokens: 162379 input, 14354 output (176733 total).

Estimated provider cost: \$0.451229 USD estimated.

- interpretation (openai:gpt-6.1-sol): 3920 input, 2330 output tokens; completed; \$0.028041 estimated.
- ai_general_reasoning (openai:gpt-6.1-sol): 25980 input, 1459 output tokens; completed; \$0.065184 estimated.
- additional_literature_search (openai:gpt-6.1-sol): 8218 input, 87 output tokens; completed; \$0.017306 estimated.
- source_selection (openai:gpt-6.1-sol): 14304 input, 142 output tokens; completed; \$0.030028 estimated.
- evidence_extraction (openai:gpt-6.1-sol): 16113 input, 1310 output tokens; completed; \$0.045326 estimated.
- experiment_planning (openai:gpt-6.1-sol): 3556 input, 1862 output tokens; completed; \$0.022171 estimated.
- experiment_code (openai:gpt-6.1-sol): 16237 input, 4247 output tokens; completed; \$0.074944 estimated.

- experiment_execution (openai:gpt-6.1-sol): 23800 input, 710 output tokens; completed; \$0.051698 estimated.
- evidence_synthesis (openai:gpt-6.1-sol): 24718 input, 1212 output tokens; completed; \$0.058495 estimated.
- confidence_estimation (openai:gpt-6.1-sol): 25533 input, 995 output tokens; completed; \$0.058035 estimated.